

mini_copilot

Guide complet du projet

Un assistant RAG sur des supports de cours : de l'API FastAPI au déploiement, de la recherche hybride à l'évaluation mesurée

FastAPI · Mistral · Postgres + pgvector · Neon · Docker · Fly.io · évaluation sur golden set

État au commit 5056403 — 15/09/2026

Sommaire

1. Vue d'ensemble	5
1.1 Ce que fait le projet	5
1.2 L'état actuel en chiffres	5
1.3 La pile technique et pourquoi	5
1.4 Comment lire ce guide	6
2. Architecture	6
2.1 Le trajet d'une requête	6
2.2 Le trajet d'un document (ingestion)	6
2.3 Rôle de chaque fichier	6
2.4 Principes de conception transversaux	7
3. Chronologie du projet	7
4. Étape 1 — Le squelette FastAPI	8
4.1 L'objectif : un service trivial, mais de production	8
4.2 La couche HTTP : main.py	8
4.3 La configuration : config.py	8
4.4 L'appel au modèle en HTTP brut	9
4.5 Trois garde-fous posés dès le départ	9
5. Le premier débogage : une « réponse vide »	9
5.1 Le vrai coupable : le quota est par modèle	9
5.2 Des serveurs fantômes sur le port 8000	10
5.3 L'encodage de la console Windows	10
6. Conteneurisation	10
6.1 Le Dockerfile, ligne par ligne	10
6.2 Docker Compose : l'API et sa base	10
6.3 Deux pièges rencontrés	10
7. Sécurité	11
7.1 Pourquoi protéger /chat	11
7.2 La clé partagée X-API-Key	11
7.3 Les secrets et git	11
8. Déploiement sur Fly.io	12
8.1 La configuration fly.toml	12
8.2 La procédure	12
8.3 Vérifier un déploiement	12
9. Étape 2 — Le RAG	13
9.1 Le corpus et sa réalité	13
9.2 Lecture des documents : parsers.py	13

9.3 Le découpage : chunking.py	13
9.4 Les embeddings : embeddings.py	14
9.5 Le schéma : db.py	14
9.6 L'ingestion : ingest.py	14
9.7 Quatre pièges de démarrage	14
10. La recherche hybride	15
10.1 Ce que mesure la distance cosinus	15
10.2 Le seuil de distance ne détecte pas le hors-sujet	15
10.3 Le cas Welsh & Powell	15
10.4 La requête SQL	15
10.5 La fusion RRF expliquée	16
11. La génération	16
11.1 Le prompt système	16
11.2 Les sources dans la réponse	16
12. La production avec Neon	17
12.1 Le raccordement	17
12.2 Deux défauts révélés par la première vraie question	17
13. Étape 4 — L'évaluation	17
13.1 Pourquoi des métriques sans LLM (et pas Ragas)	17
13.2 Les métriques	17
13.3 Le golden set	18
13.4 Mesurer	18
13.5 La baseline	19
13.6 Ce que les mesures ont appris sur les mesures	19
14. La réécriture de requête	19
14.1 Le principe	19
14.2 Trois garde-fous	20
14.3 Les résultats	20
14.4 La décision et son effet en production	20
15. Exploiter le projet au quotidien	21
15.1 Démarrer en local	21
15.2 Aide-mémoire des commandes	21
15.3 Ajouter des documents au corpus	21
15.4 Enrichir le golden set	21
16. Dépannage	22
17. Les leçons transversales	22
18. La suite	23

18.1 Étape 3 — Observabilité avec Langfuse (recommandée)	23
18.2 Les étapes suivantes	23
18.3 Les limites connues à traiter	24
Annexe A — Glossaire	24
Annexe B — Arborescence du dépôt	24

1. Vue d'ensemble

1.1 Ce que fait le projet

mini_copilot est un assistant qui répond à des questions à partir de supports de cours, et uniquement à partir d'eux. On lui envoie une question en HTTP ; il retrouve les passages pertinents dans une base vectorielle, les donne à un modèle de langage avec l'interdiction de répondre autre chose, et renvoie la réponse accompagnée de ses sources (document et page). Quand les cours ne contiennent pas la réponse, il le dit au lieu d'inventer.

Le projet a été conçu pour **apprendre la chaîne complète autour d'un LLM** — le versant MLOps — plutôt que la modélisation : le modèle est consommé via une API, et tout l'effort porte sur l'ingestion, la recherche, l'orchestration, la sécurité, le déploiement et surtout la mesure.

1.2 L'état actuel en chiffres

Élément	Valeur
Service en ligne	https://mini-copilot.fly.dev (Fly.io, région Paris)
Base de production	Neon (PostgreSQL 18.6, pgvector 0.8.6, Francfort)
Corpus indexé	15 documents (6 PDF, 9 PPTX) → 1076 chunks
Modèle de génération	ministral-8b-latest
Modèle de réécriture	ministral-3b-latest (activé en production)
Modèle d'embeddings	mistral-embed (vecteurs de dimension 1024)
Qualité du retrieval	Hit@5 = 1.000, MRR = 0.988 sur 58 questions
Tests automatisés	66 tests, sans réseau ni base
Code	≈ 2 650 lignes Python (app, eval, tests)

1.3 La pile technique et pourquoi

Brique	Choix	Raison
API	FastAPI + Pydantic v2	Standard pour servir de l'IA en Python ; validation des entrées et documentation interactive gratuites.
LLM	API Mistral, appelée en HTTP brut (httpx)	Voir qu'un appel LLM n'est qu'un POST JSON ; aucune magie de SDK.
Base vectorielle	Postgres + pgvector	Une seule infrastructure : SQL classique et recherche vectorielle au même endroit.
Recherche	Hybride : vecteurs + mots-clés (tsvector), fusion RRF	Le vectoriel comprend le sens, les mots-clés ne ratent pas un nom propre.
Conteneur	Docker, Docker Compose	Même image en local et en production : la fin du « ça marche chez moi ».
Hébergement	Fly.io (build distant, scale-to-zero)	Déploiement du Dockerfile tel quel, coût nul au repos.
Base en production	Neon, offre gratuite	pgvector inclus, joignable depuis le PC pour l'ingestion, sans tunnel.
Qualité	pytest, ruff, GitHub Actions	Tests et lint rejoués à chaque push (la CI s'activera avec un dépôt distant).
Évaluation	Golden set + métriques calculées à la main	Déterministe, gratuit, adapté à la décision à prendre.

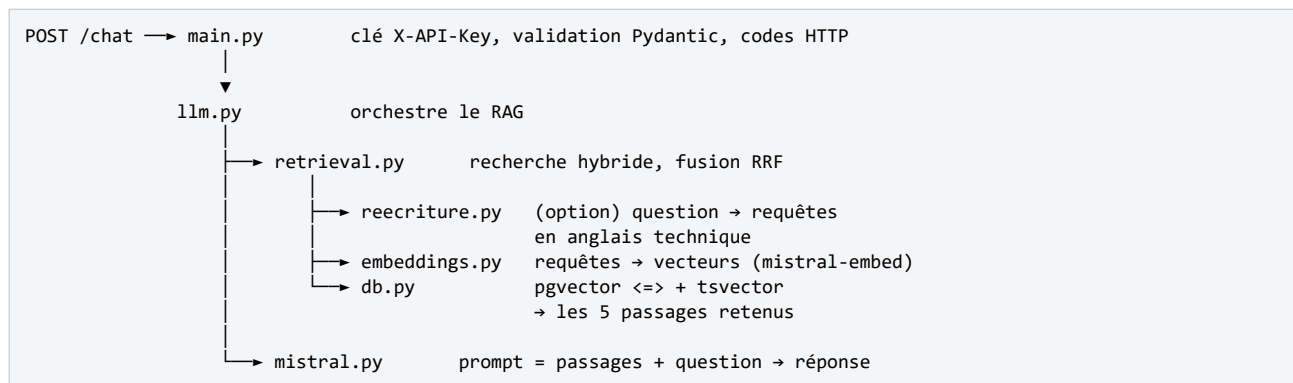
1.4 Comment lire ce guide

Les chapitres suivent l'ordre de construction du projet, parce que chaque choix s'explique par ce qui le précède. Chaque chapitre mélange trois choses : **ce qui a été construit**, **pourquoi**, et **ce qui a cassé** en chemin. Les encadrés verts résument les leçons, les orangés signalent un piège, les rouges une erreur réellement commise pendant le projet — gardée ici parce qu'elle apprend davantage qu'un parcours sans faute.

2. Architecture

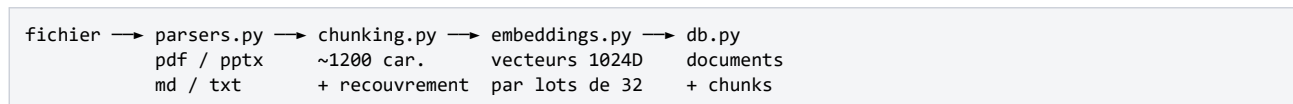
2.1 Le trajet d'une requête

Une question posée à `/chat` traverse les couches suivantes. Chaque module a une responsabilité unique, ce qui permet de changer une étape sans toucher aux autres.



2.2 Le trajet d'un document (ingestion)

L'ingestion tourne hors ligne, en ligne de commande, depuis le PC.



2.3 Rôle de chaque fichier

Fichier	Rôle
<code>app/main.py</code>	Couche HTTP : authentification, validation, traduction des pannes en codes HTTP. Aucune logique métier.
<code>app/llm.py</code>	Orchestration : chercher les passages, construire le prompt, appeler le modèle, renvoyer texte et sources.
<code>app/retrieval.py</code>	Recherche hybride en SQL, fusion RRF, recherche multi-requêtes.
<code>app/reecriture.py</code>	Réécriture de la question en requêtes anglaises.
<code>app/embeddings.py</code>	Texte → vecteurs, par lots, avec back-off sur quota.
<code>app/mistral.py</code>	Client HTTP partagé, appel chat, messages d'erreur actionnables.
<code>app/parsers.py</code>	Un lecteur par format de fichier et normalisation du texte.
<code>app/chunking.py</code>	Découpage du texte en passages indexables.
<code>app/ingest.py</code>	Pipeline d'ingestion rejouable (empreinte SHA-256).
<code>app/db.py</code>	Schéma, pool de connexions, amorçage de la base.
<code>app/config.py</code>	Configuration et secrets lus depuis l'environnement.
<code>app/boucle.py, run.py</code>	Correctif de boucle d'événements pour Windows.

Fichier	Rôle
eval/	Golden set, métriques, mesures, synthèse des répétitions.
tests/	66 tests sans réseau.
Dockerfile, docker-compose.yml	Image de production ; API + base en local.
fly.toml	Configuration Fly.io (réglages non secrets).
.github/workflows/ci.yml	Lint, tests et build Docker à chaque push.

2.4 Principes de conception transversaux

- **Une nouveauté par étape.** On met en ligne un service trivial avant d'ajouter le RAG, pour ne jamais déboguer deux inconnues à la fois.
- **Aucun secret dans le code.** Tout vient de l'environnement ; `.env` et `.env.prod` sont ignorés par git.
- **Échouer vite et fort.** Variable manquante, base injoignable : l'app refuse de démarrer avec un message qui nomme la cause.
- **Une panne se diagnostique depuis son message.** Code HTTP juste (502 quand la faute est en amont), type d'exception nommé, marche à suivre.
- **Mesurer avant de décider.** Seuil de distance, recherche hybride, réécriture : chaque choix du retrieval a été tranché par une mesure.

3. Chronologie du projet

L'historique git raconte le projet. Chaque message de commit explique le *pourquoi* du changement, pas seulement le *quoi* : `git log` est une documentation à part entière.

Commit	Date	Étape
06c43e4	04/09	Étape 1 : squelette FastAPI avec <code>/chat</code> vers Mistral
4c76430	04/09	Authentification par clé partagée, configuration Fly.io
f0576fd	04/09	Premier déploiement en ligne sur Fly.io
071b9e2	07/09	Étape 2 : RAG multi-format sur pgvector
d1e56f7	07/09	Recherche hybride : vectoriel + mots-clés, fusion RRF
e22883e	07/09	Échec rapide et lisible si la base est injoignable
7df0e69	07/09	Préparation du raccordement à Neon
25d09f4	07/09	Règle <code>.gitignore</code> élargie après un quasi-commit de secret
2ec8ef4	07/09	Délai LLM à 90 s, exceptions muettes rendues lisibles
b82a66a	15/09	Étape 4 : évaluation du retrieval, baseline
4f5051f	15/09	Réécriture de requête et évaluation traçable
0649b60	15/09	Premières mesures répétées (golden set v2)
e080f3f	15/09	Mesures répétées terminées : gain établi
5056403	15/09	Réécriture activée en production (<code>ministral-3b</code>)

4. Étape 1 — Le squelette FastAPI

4.1 L'objectif : un service trivial, mais de production

La première version n'a pas de RAG : un endpoint `/chat` reçoit une question et l'envoie directement au modèle. C'est délibéré. Le squelette établit tout ce qui ne changera plus — validation, configuration, gestion d'erreurs, tests, conteneur — et il est mis en ligne immédiatement, pendant qu'il est encore trivial.

4.2 La couche HTTP : main.py

Les modèles Pydantic décrivent le contrat de l'API. FastAPI s'en sert pour valider les entrées (une question vide renvoie `422` sans une ligne de code) et pour générer la documentation interactive sur `/docs`.

```
class ChatRequest(BaseModel):
    question: str = Field(..., min_length=1, max_length=4000)

class Source(BaseModel):
    document: str
    page: int | None
    distance: float

class ChatResponse(BaseModel):
    answer: str
    model: str
    sources: list[Source]
```

La route `/chat` ne contient aucune logique métier : elle délègue à `generate_answer()` et traduit les pannes.

```
try:
    reponse = await generate_answer(request.question)
except LLMError as exc:
    # 502 et pas 500 : la faute vient d'un service en amont, pas de nous.
    raise HTTPException(status_code=502, detail=str(exc)) from exc
```

Pourquoi 502 plutôt que 500

Un `500` dit « mon code a planté ». Un `502 Bad Gateway` dit « un service dont je dépends a échoué ». Quand Mistral tombe, ce n'est pas un bug de l'application. Avec plusieurs appels enchaînés (réécriture, embeddings, génération), savoir *qui* a échoué devient vital.

4.3 La configuration : config.py

`pydantic-settings` lit les variables d'environnement (et le fichier `.env` en local) et les valide au démarrage. Les champs sans valeur par défaut sont obligatoires : si `MISTRAL_API_KEY` ou `API_KEY` manque, l'application refuse de démarrer au lieu d'échouer à la première requête.

Variable	Obligatoire	Défaut	Rôle
<code>MISTRAL_API_KEY</code>	oui	—	Clé de l'API Mistral
<code>API_KEY</code>	oui	—	Clé du service, protège <code>/chat</code>
<code>DATABASE_URL</code>	non	<code>postgresql://rag:rag@localhost:5433/rag</code>	Base vectorielle
<code>MISTRAL_MODEL</code>	non	<code>ministral-8b-latest</code>	Modèle de génération
<code>MISTRAL_EMBED_MODEL</code>	non	<code>mistral-embed</code>	Modèle d'embeddings
<code>MISTRAL_BASE_URL</code>	non	<code>https://api.mistral.ai/v1</code>	URL de l'API
<code>REQUEST_TIMEOUT_SECONDS</code>	non	<code>90</code>	Délai des appels LLM
<code>REECRITURE_REQUETES</code>	non	<code>false</code>	Active la réécriture de requête
<code>MISTRAL_REECRITURE_MODEL</code>	non	<code>ministral-8b-latest</code>	Modèle de réécriture

4.4 L'appel au modèle en HTTP brut

L'appel à Mistral est écrit avec `httpx`, sans SDK : un POST sur `/chat/completions`, un header d'authentification, un corps JSON. Un client HTTP est partagé pour réutiliser les connexions ; il est fermé proprement à l'arrêt de l'application grâce au `lifespan` de FastAPI.

```
payload = {"model": modele, "messages": messages}
headers = {"Authorization": f"Bearer {settings.mistral_api_key}"}
response = await get_client().post("/chat/completions", json=payload, headers=headers)
contenu = response.json()["choices"][0]["message"]["content"]
```

4.5 Trois garde-fous posés dès le départ

- **Une réponse vide n'est jamais un succès.** Si le fournisseur renvoie `200` avec un contenu vide, l'appel lève une erreur explicite. Un `200` mensonger est le bug qu'on cherche pendant des heures.
- **Les erreurs du fournisseur deviennent actionnables.** Un `429` indique que le quota est par modèle et qu'il faut changer `MISTRAL_MODEL` ; un `401` pointe vers la clé.
- **Les tests ne touchent jamais le réseau.** Le LLM est remplacé par un double. Un test qui dépend d'une API externe est un pari qui casserait la CI au premier hoquet du fournisseur.

5. Le premier débogage : une « réponse vide »

Dès le premier essai, `/chat` semblait renvoyer une réponse vide. Ce débogage a révélé trois problèmes distincts — et une erreur de diagnostic.

5.1 Le vrai coupable : le quota est par modèle

L'API répondait `429 Rate limit exceeded` avec l'en-tête `x-ratelimit-limit-req-minute: 0`. Le premier diagnostic — « le compte n'est pas activé » — était **faux**. Ce qui a tranché : `GET /v1/models` répondait `200` (la clé était donc valide), et un balayage des modèles a montré que la limite dépendait du modèle.

Modèle	Statut	Requêtes/min
<code>mistral-large-latest</code>	403, hors abonnement	—
<code>mistral-medium-latest</code>	429	0
<code>mistral-small-latest</code>	429 (défaut initial du projet)	0
<code>magistral-small-latest</code>	429	0
<code>ministral-8b-latest</code>	OK — génération	188
<code>open-mistral-nemo</code>	OK	188
<code>open-mistral-7b</code>	OK	188
<code>ministral-3b-latest</code>	OK — réécriture	750
<code>mistral-embed</code>	OK — embeddings	60

Erreur commise : conclure avant d'isoler

J'ai d'abord envoyé l'utilisateur activer son compte par SMS, alors que le compte était actif. La bonne méthode : isoler ce qui échoue (la clé ? le compte ? le modèle ?) avec un test qui ne peut échouer que pour une seule raison — ici, lister les modèles puis tester chacun.

5.2 Des serveurs fantômes sur le port 8000

Trois processus écoutaient déjà le port 8000. Une requête pouvait donc atteindre un ancien serveur exécutant du vieux code, et un nouvel `uvicorn` pouvait échouer à démarrer sans que ce soit visible.

```
netstat -ano | findstr :8000
```

Plus d'une ligne `LISTENING` signifie que plusieurs serveurs se disputent le port. On ferme les anciens (`taskkill /PID <pid> /F`) ou on lance sur un autre port.

5.3 L'encodage de la console Windows

Un script qui affichait une réponse du modèle a planté sur `UnicodeEncodeError` : la console Windows est en cp1252 et ne sait pas écrire un emoji. L'application n'est pas concernée (FastAPI renvoie du JSON UTF-8), mais tout script local doit être lancé avec `PYTHONIOENCODING=utf-8`.

6. Conteneurisation

6.1 Le Dockerfile, ligne par ligne

```
FROM python:3.13-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app

# Les dépendances d'abord, le code ensuite : tant que requirements.txt ne
# change pas, Docker réutilise le cache et le rebuild prend 2 secondes.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app

# Ne jamais tourner en root dans un conteneur.
RUN useradd --create-home appuser
USER appuser

EXPOSE 8000
CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

- `python:3.13-slim` : image légère ; l'image finale construite par Fly pèse environ 51 Mo compressée.
- **Ordre des COPY** : la couche des dépendances n'est reconstruite que si `requirements.txt` change.
- **Utilisateur non-root** : une faille dans l'application ne donne pas les droits root dans le conteneur.
- `${PORT:-8000}` : la plateforme peut imposer le port à l'exécution.
- `.dockerignore` : `.env`, tests, `eval/` et `.git` n'entrent jamais dans l'image.

6.2 Docker Compose : l'API et sa base

`docker-compose.yml` démarre l'API et un Postgres avec pgvector (`pgvector/pgvector:pg17`). L'API attend que la base soit saine (`depends_on + condition: service_healthy`). Dans le réseau Compose, la base s'appelle `db` ; elle est aussi exposée sur le port 5433 de la machine pour que l'ingestion lancée hors conteneur puisse s'y connecter.

6.3 Deux pièges rencontrés

Le healthcheck qui échouait toujours

Il interrogeait `http://localhost:8000/health`. Dans le conteneur, `localhost` résout d'abord en `:::1` (IPv6), alors qu'`uvicorn` n'écoute qu'en IPv4 : connexion refusée, conteneur marqué `unhealthy` en permanence. Correctif : `127.0.0.1`.

Le build « réussi » qui ne l'était pas

La commande `docker build ... | tail -30` a renvoyé le code de sortie `0...` celui de `tail`. Docker Desktop n'était même pas démarré. Dans un pipeline shell, le code de sortie est celui de la dernière commande : il faut `set -o pipefail` ou lire `${PIPESTATUS[0]}`.

7. Sécurité

7.1 Pourquoi protéger /chat

Une fois en ligne, un `/chat` sans authentification est un proxy LLM gratuit offert à tout Internet, payé avec le quota du propriétaire. La protection a été ajoutée **avant** le premier déploiement, pas après.

7.2 La clé partagée X-API-Key

```
api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)

def require_api_key(provided: str | None = Security(api_key_header)) -> None:
    expected = get_settings().api_key
    if provided is None or not secrets.compare_digest(provided, expected):
        raise HTTPException(status_code=401, detail="Cle API manquante ou invalide")
```

- `secrets.compare_digest` et pas `==` : la comparaison prend le même temps quelle que soit la clé fournie, donc le temps de réponse ne révèle pas la clé caractère par caractère.
- **Fail-closed** : `API_KEY` est obligatoire ; un service exposé ne peut pas démarrer sans authentification parce qu'une variable a été oubliée.
- `/health` reste public : Fly.io l'interroge sans header, et il ne fait aucun appel externe.
- **Un 401 n'atteint jamais Mistral** : vérifié dans les logs et verrouillé par un test. Sinon un attaquant viderait le quota même en échouant.

Deux clés distinctes coexistent, à ne pas confondre : `MISTRAL_API_KEY` (Mistral te facture) et `API_KEY` (protège ton service, générée avec `secrets.token_urlsafe(32)`).

7.3 Les secrets et git

`.env` et `.env.prod` sont dans `.gitignore`. Avant chaque commit du projet, l'index a été vérifié : on recherche la valeur littérale de chaque clé dans `git diff --cached`.

Incident évité : un mot de passe dans l'index

Un fichier nommé `env.prod` (sans le point) contenant la chaîne de connexion Neon, mot de passe compris, s'est retrouvé dans l'index : la règle `.env.prod` ne le couvrait pas. Il a été retiré avant tout commit et la règle élargie en `*env.prod`. Un secret entré dans l'historique git ne s'efface pas : il se révoque.

8. Déploiement sur Fly.io

8.1 La configuration fly.toml

```
app = "mini-copilot"
primary_region = "cdg" # Paris

[env]
MISTRAL_MODEL = "ministral-8b-latest"
MISTRAL_BASE_URL = "https://api.mistral.ai/v1"
REECRITURE_REQUETES = "true"
MISTRAL_REECRITURE_MODEL = "ministral-3b-latest"

[http_service]
internal_port = 8000
force_https = true
auto_stop_machines = "stop"
auto_start_machines = true
min_machines_running = 0

[[http_service.checks]]
interval = "30s"
timeout = "5s"
method = "GET"
path = "/health"
```

- **Build distant** : Fly construit l'image sur ses serveurs ; Docker Desktop n'a pas besoin de tourner.
- **[env] contre secrets** : les réglages non secrets sont versionnés dans `fly.toml` ; les secrets passent par `fly secrets set` et vivent chiffrés chez Fly.
- **Scale-to-zero** : les machines s'arrêtent faute de trafic et redémarrent à la requête suivante — zéro coût au repos, quelques secondes de réveil.
- **Deux machines** : Fly en crée deux par défaut pour la haute disponibilité ; `fly scale count 1` en garde une seule.

8.2 La procédure

```
fly auth login
fly apps create mini-copilot --org personal
fly secrets set MISTRAL_API_KEY=... API_KEY=... DATABASE_URL=... --stage
fly deploy --remote-only
```

Les secrets ont été posés par substitution shell à partir de `.env` et `.env.prod`, pour qu'aucune valeur n'apparaisse en clair dans la conversation ni dans l'historique du terminal. `fly secrets list` n'affiche que des empreintes.

8.3 Vérifier un déploiement

Test	Attendu
GET /health	200 {"status":"ok"}
POST /chat sans clé	401
POST /chat avec une mauvaise clé	401
POST /chat avec la bonne clé	200, réponse et sources
http:// au lieu de https://	301 vers HTTPS
fly status	machines saines, checks au vert

9. Étape 2 — Le RAG

9.1 Le corpus et sa réalité

Le corpus réel — des supports de cours — a imposé ses contraintes avant toute ligne de code d'ingestion. Il a été inspecté d'abord.

- **9 fichiers sur 16 étaient des .pptx**, pas des PDF : le multi-format était le cas de base, pas une option.
- **Le ZIP contenait les 9 mêmes .pptx** : l'indexer aurait dupliqué chaque passage et évincé le reste des résultats. Il est ignoré.
- **Tout le corpus est en anglais** — ce qui deviendra le problème central de l'étape 4, les questions étant posées en français.
- **Un livre de 254 pages pèse 62 % de la base** (667 chunks sur 1076).

9.2 Lecture des documents : parsers.py

Lire un fichier et découper son texte sont deux étapes séparées. Ajouter un format revient à écrire une fonction et l'inscrire dans un registre, sans toucher au découpage.

```
PARSERS = {
    ".pdf": lire_pdf,      # pypdf, page par page
    ".pptx": lire_pptx,    # python-pptx : zones de texte, tableaux, notes
    ".md": lire_texte,
    ".txt": lire_texte,
    ".rst": lire_texte,
}
```

Chaque lecteur renvoie des couples (**numéro de page**, **texte**). Le numéro est conservé jusqu'à la réponse : c'est ce qui permet de citer « page 12 ». Pour les **.pptx**, les **tableaux** et les **notes du présentateur** sont extraits — sur les diapositives de Dijkstra, c'est le tableau qui porte le contenu.

La normalisation du texte

Défaut	Conséquence	Correction
Ligatures des PDF LaTeX (fi = U+FB01)	« final » ne correspond jamais à une recherche sur « final »	Normalisation NFKC + remplacement des ligatures
Octets NUL (0x00)	Postgres refuse l'insertion (DataError)	Suppression des caractères de contrôle sauf \n et \t
Césure de fin de ligne	« opti- ln misation » coupé en deux	Recollage
Espaces insécables	Tokens parasites	Remplacés par des espaces

9.3 Le découpage : chunking.py

C'est la décision qui fait la qualité d'un RAG. Trop gros, le passage utile est noyé et le vecteur devient flou ; trop petit, un théorème est séparé de sa démonstration.

Paramètre	Valeur	Justification
TAILLE_CIBLE	1200 caractères	≈ 300 tokens : un raisonnement complet, encore précis
RECOUVREMENT	180 caractères	Une idée coupée en deux n'est perdue par aucun des deux morceaux
TAILLE_MINIMALE	80 caractères	En dessous : un titre ou un numéro de page, du bruit

Le découpage suit les frontières naturelles : paragraphes, puis phrases. Un **filet de sécurité** coupe aux espaces les blocs qui n'ont ni ligne vide ni ponctuation — des légendes de figures mathématiques produisaient sinon des chunks de 3259 caractères, près de trois fois la cible.

9.4 Les embeddings : embeddings.py

- **Par lots de 32** : 1076 chunks en un appel chacun tiendraient 18 minutes au quota de 60 requêtes/minute ; en lots, environ une minute.
- **Back-off exponentiel sur 429** : 2 s, 4 s, 8 s, 16 s, 32 s. Un 429 est une invitation à ralentir, pas une erreur fatale.
- **Tri par index** des vecteurs renvoyés : un décalage associerait chaque texte au vecteur d'un autre, bug invisible jusqu'à ce que le retrieval déraile.

9.5 Le schéma : db.py

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE documents (
  id BIGSERIAL PRIMARY KEY, source TEXT NOT NULL UNIQUE, titre TEXT NOT NULL,
  format TEXT NOT NULL, empreinte TEXT NOT NULL,
  ingere_le TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE chunks (
  id BIGSERIAL PRIMARY KEY,
  document_id BIGINT NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
  position INT NOT NULL, page INT, contenu TEXT NOT NULL,
  embedding vector(1024) NOT NULL,
  tsv tsvector GENERATED ALWAYS AS (to_tsvector('simple', contenu)) STORED
);

CREATE INDEX chunks_tsv_idx ON chunks USING gin (tsv);
CREATE INDEX chunks_embedding_idx ON chunks USING hnsw (embedding vector_cosine_ops);
```

- **SQL écrit à la main**, sans ORM : la recherche vectorielle est du SQL très particulier, qu'il faut voir pour le comprendre.
- **vector(1024)** : Postgres refuse tout vecteur d'une autre dimension, donc un mélange accidentel de modèles.
- **tsv générée** : recalculée par Postgres à chaque écriture, impossible à désynchroniser du texte.
- **Index HNSW créé après l'insertion** : le construire une fois à la fin est bien plus rapide qu'au fil des inserts.

9.6 L'ingestion : ingest.py

```
python -m app.ingest "C:/chemin/vers/corpus"
python -m app.ingest "C:/chemin/vers/corpus" --force
```

- **Rejouable** : une empreinte SHA-256 par fichier ; un fichier inchangé est ignoré, sans consommer de quota ni dupliquer de chunks.
- **Tout ou rien** : l'insertion d'un document se fait dans une transaction ; une réingestion remplace (via **ON DELETE CASCADE**) au lieu de dupliquer.
- **Limite connue** : l'ingestion ne supprime jamais un document dont le fichier a disparu du dossier.

9.7 Quatre pièges de démarrage

Symptôme	Cause	Correctif
<code>PoolTimeout</code> au démarrage	Le pool enregistre le type <code>vector</code> à chaque connexion ; l'extension n'existant pas encore, aucune connexion ne s'ouvre	Créer le schéma sur une connexion directe, avant d'ouvrir le pool
<code>InterfaceError: ProactorEventLoop</code>	psycpgp asynchrone refuse la boucle par défaut de Windows	<code>app/boucle.py</code> et <code>python run.py</code> en local

Symptôme	Cause	Correctif
<code>operator does not exist: vector <=> double precision[]</code>	Une liste Python part en tableau de flottants ; l'INSERT convertit, l'opérateur non	Envoyer <code>pgvector.Vector(...)</code>
Démarrage bloqué indéfiniment	Base injoignable sans délai de connexion	<code>connect_timeout=10</code> et message clair, mot de passe masqué

10. La recherche hybride

10.1 Ce que mesure la distance cosinus

Chaque passage et chaque question sont des vecteurs de 1024 dimensions. L'opérateur `<=>` de `pgvector` calcule la distance cosinus : 0 pour deux directions identiques, plus pour des sens éloignés. Trier par distance, c'est classer les passages du sens le plus proche au plus lointain.

10.2 Le seuil de distance ne détecte pas le hors-sujet

L'idée naturelle — rejeter les passages au-delà d'une distance — a été mesurée plutôt que supposée, sur 10 questions du cours et 8 hors-sujet.

Groupe	min	médiane	max
Questions du cours	0.145	0.219	0.286
Questions hors-sujet	0.272	0.303	0.396

Les deux distributions se chevauchent

Aucun seuil ne sépare les deux groupes : la distance mesure la proximité thématique, pas la capacité à répondre. Le seuil (**0.35**) reste un filtre grossier réglé pour ne jamais perdre un bon passage ; c'est le **prompt** qui interdit au modèle de répondre hors des passages fournis.

10.3 Le cas Welsh & Powell

La question « Quelles sont les étapes de l'algorithme de Welsh & Powell ? » ne remontait **pas** le cours sur Welsh & Powell. La tournure « étapes de l'algorithme » dominait le vecteur et attirait le pseudo-code de Floyd-Warshall, pendant que le nom propre ne pesait presque rien. Reformulée en « algorithme Welsh Powell coloration », la même recherche trouvait le bon passage à 0.097.

C'est la faiblesse connue de la recherche purement vectorielle : les termes rares et les noms propres. La recherche par mots-clés, elle, ne comprend rien mais ne rate jamais un nom exact. Les deux se complètent.

10.4 La requête SQL

```
WITH vectoriel AS (
  SELECT c.id, row_number() OVER (ORDER BY c.embedding <=> %(v)s) AS rang
  FROM chunks c
  WHERE c.embedding <=> %(v)s <= %(dmax)s
  ORDER BY c.embedding <=> %(v)s
  LIMIT %(k)s
),
mots_cles AS (
  SELECT c.id, row_number() OVER (ORDER BY ts_rank(c.tsv, q) DESC) AS rang
  FROM chunks c, websearch_to_tsquery('simple', %(q)s) q
  WHERE c.tsv @@ q
  ORDER BY ts_rank(c.tsv, q) DESC
  LIMIT %(k)s
),
fusion AS (
  SELECT COALESCE(v.id, m.id) AS id,
```

```

        COALESCE(1.0 / (%(krrf)s + v.rang), 0)
        + COALESCE(1.0 / (%(krrf)s + m.rang), 0) AS score
    FROM vectoriel v FULL OUTER JOIN mots_cles m ON m.id = v.id
)
SELECT c.id, c.contenu, d.source, d.titre, c.page, c.embedding <=> %(v)s AS distance
FROM fusion f JOIN chunks c ON c.id = f.id JOIN documents d ON d.id = c.document_id
ORDER BY f.score DESC, distance ASC
LIMIT %(n)s

```

10.5 La fusion RRF expliquée

La **Reciprocal Rank Fusion** combine des classements, pas des scores. Chaque moteur vote pour ses passages avec $1 / (K + \text{rang})$, K valant 60. Comparer une distance cosinus à un **ts_rank** n'aurait aucun sens : ce sont deux grandeurs sans commune mesure. Les rangs, eux, se comparent.

Passage	Rang vectoriel	Rang mots-clés	Score RRF
A	1	absent	$1/61 = 0.0164$
B	3	3	$1/63 + 1/63 = \mathbf{0.0317}$
C	absent	1	$1/61 = 0.0164$

B, troisième dans les deux listes, passe devant les premiers de chaque liste : être retrouvé par les deux moteurs est le signal le plus fiable. La constante 60 amortit l'écart entre les premières places.

- **Configuration simple** et non **french** : le corpus mêle français et anglais, et la racinisation abîmerait les termes rares recherchés.
- **25 candidats par moteur** avant fusion : un passage 12^e en vectoriel et 3^e en mots-clés doit pouvoir remonter.

11. La génération

11.1 Le prompt système

Tu es un assistant qui répond à partir des supports de cours fournis, et UNIQUEMENT à partir d'eux.

Règles :

- Appuie chaque affirmation sur les passages numérotés ci-dessous, et cite la source entre crochets, par exemple [2].
- Si les passages ne contiennent pas de quoi répondre, dis-le franchement : « Je ne trouve pas la réponse dans tes supports de cours. » N'utilise alors PAS tes connaissances générales.
- Ne complète jamais un passage par ce que tu crois savoir par ailleurs.
- Réponds en français, de façon claire et pédagogique.

Les passages sont numérotés dans le message utilisateur, chacun avec sa citation (« VII. Shortest paths (p. 10) »). Si aucun passage n'est retenu, un second prompt ordonne un refus direct.

Pourquoi c'est le prompt qui porte le garde-fou

Un RAG qui invente est pire qu'inutile : il est crédible **et** faux. Vérifié : « Quelle est la recette de la tarte aux pommes ? » obtient un refus explicite alors que cinq passages ont bien été remontés.

11.2 Les sources dans la réponse

```

{
  "answer": "L'algorithme de Dijkstra permet de calculer les plus courts chemins [1]...",
  "model": "ministral-8b-latest",
  "sources": [
    {"document": "VII. Shortest paths", "page": 10, "distance": 0.1445},
    {"document": "VII. Shortest paths", "page": 53, "distance": 0.1465}
  ]
}

```

}

Exposer les sources n'est pas cosmétique : sans elles, l'utilisateur doit croire le modèle sur parole.

12. La production avec Neon

12.1 Le raccordement

- **Choix de Neon** : gratuit, pgvector inclus, et joignable depuis Internet — l'ingestion lancée depuis le PC écrit directement dans la base, sans tunnel.
- **Le mot de passe ne transite pas par la conversation** : il est placé dans `.env.prod` (ignoré par git) et lu par substitution shell.
- **Schéma créé, 1076 chunks réingérés, retrieval vérifié sur Neon** avant de toucher à la production.

12.2 Deux défauts révélés par la première vraie question

Défaut	Cause	Correctif
La question sur Welsh & Powell échoue en production	<code>httpx.ReadTimeout</code> : 30 s ne suffisent pas pour générer une réponse longue avec 5 passages	Délai porté à 90 s
Message d'erreur « Appel au LLM impossible : »	<code>httpx.ReadTimeout</code> a une représentation textuelle vide	Le type de l'exception est nommé dans le message

Leçon

Une panne doit se diagnostiquer depuis le message d'erreur, pas en allant lire les logs du serveur.

13. Étape 4 — L'évaluation

13.1 Pourquoi des métriques sans LLM (et pas Ragas)

Si le golden set indique que la réponse est dans **III. Breadth first search**, vérifier que ce document remonte est une comparaison. Les métriques de retrieval se calculent donc sans modèle : gratuites, et avec un calcul déterministe.

Ragas sert surtout à juger la *génération* avec un LLM qui découpe la réponse en affirmations puis les vérifie. Sur ce compte, les seuls modèles disponibles font 3 à 8 milliards de paramètres : un juge de cette taille donne des scores bruités et des sorties JSON illisibles. L'évaluation de la génération reste à faire, avec un juge adapté.

13.2 Les métriques

Métrique	Définition	Ce qu'elle révèle
Hit@5	1 si au moins un passage pertinent est dans les 5 premiers	Le bon contenu arrive-t-il sous les yeux du modèle ?
MRR	Moyenne de 1/rang du premier passage pertinent	Distingue trouvé en 1 ^{re} position (1.0) de trouvé en 5 ^e (0.2)
Précision@5	Part des 5 passages qui sont pertinents	L'encombrement du contexte par du hors-sujet
Hit@5 page	Hit@5 au niveau de la page étiquetée	La précision de la localisation, pour les citations

Exemple : les passages renvoyés sont pertinents aux rangs 3 et 5 (`[non, non, oui, non, oui]`). Hit@5 = 1, RR = 1/3 = 0.333, Précision@5 = 2/5 = 0.4.

13.3 Le golden set

`eval/golden_set.yaml` contient **63 questions**. Le corpus étant en anglais, les catégories isolent l'effet de la langue.

Catégorie	Principe	Effectif	Exemple
<code>fr_proche</code>	français, terme clé proche de l'anglais	31	« algorithme de Dijkstra »
<code>fr_eloigne</code>	français, terme clé sans ressemblance	20	« parcours en largeur » / breadth-first search
<code>en</code>	groupe témoin en anglais	7	« How does breadth-first search work? »
<code>hors_sujet</code>	rien à trouver	5	« recette de la tarte aux pommes »

```
- id: bfs-fr
  question: "Comment fonctionne le parcours en largeur d'un graphe ?"
  categorie: fr_eloigne
  paire: bfs-en
  documents: ["III. Breadth first search"]
  pages: {"III. Breadth first search": [1, 2, 3]}
```

Les **paires** relient une question française à la même question en anglais : l'écart entre les deux mesure la langue, toutes choses égales par ailleurs.

Les règles d'étiquetage

- **Jamais avec le retrieval évalué** : les pages ont été localisées par recherche textuelle directe dans le texte extrait. Étiqueter avec le système qu'on mesure rendrait la mesure circulaire.
- **Une page n'est étiquetée que si elle a été lue** ; en cas de doute, seul le document est attendu.
- **Relecture symétrique** : quand le livre a été conservé dans le corpus, les quatre questions d'optimisation ont été relues contre lui, pas seulement celles où le système avait divergé (trois étiquettes ajoutées, une inchangée).
- **Correction asymétrique signalée** : la page 11 du chapitre ACP a été ajoutée après une relecture déclenchée par un résultat du système. On ne relit que là où le système conteste, jamais là où il approuve à tort : le fichier le signale.
- **Validation automatique** : identifiants uniques, catégories connues, pages rattachées à un document attendu, paires cohérentes — vérifié au chargement et par la CI.

Biais principal

Le golden set a été rédigé par celui qui construit le système. Les vraies questions de révision de l'étudiant doivent l'enrichir.

13.4 Mesurer

```
python -m eval.retrieval --nom essai --strategie hybride
python -m eval.retrieval --nom essai-rw --strategie reecriture --comparer essai
python -m eval.synthese --groupe hybride=h-r1,h-r2 --groupe reecriture=r-r1,r-r2,r-r3
```

Chaque résultat `eval/resultats/<nom>.json` enregistre :

- le **commit**, suffixé **+modifications** si le code n'était pas commité ;
- l'**empreinte du golden set** (fins de ligne normalisées) ;
- les paramètres du retrieval et la **latence** (médiane et 90^e centile) ;
- pour chaque question, les **requêtes lancées** et les passages renvoyés.

`eval.synthese` calcule moyenne et étendue [min–max] sur les répétitions et **refuse de comparer des mesures notées avec des golden sets différents** : corriger une étiquette change le score sans que le retrieval ait bougé.

13.5 La baseline

Première mesure, stratégie hybride, golden set v1 (46 questions avec réponse attendue) :

Catégorie	n	Hit@5	MRR	Précision@5	Hit@5 page
Anglais (témoin)	6	1.000	1.000	0.900	1.000
FR termes proches	22	1.000	0.898	0.782	0.952
FR termes éloignés	18	0.889	0.759	0.600	0.824
Toutes	46	0.957	0.857	0.726	0.909

Le problème de langue est réel mais localisé : deux échecs complets, tous deux sur le parcours en largeur. « Comment fonctionne le parcours en largeur d'un graphe ? » ramenait le parcours en profondeur, Floyd-Warshall et la coloration ; la même question en anglais trouvait **Breadth first search** page 1 en tête, à 0.133.

13.6 Ce que les mesures ont appris sur les mesures

Le pipeline n'est pas parfaitement reproductible

Lors d'une première série, deux exécutions identiques de la stratégie hybride ont renvoyé pour les mêmes chunks des distances différant jusqu'à $1e-3$ (79 cas sur 242) ; une quasi-égalité s'est inversée. La série suivante a donné deux exécutions identiques. Source non établie : trois appels d'embeddings rapprochés donnent des vecteurs identiques au bit près. Conclusion pratique : un écart d'un passage est du bruit, on répète avant de conclure.

La réécriture n'est pas déterministe

À température 0, trois appels identiques ne donnent presque jamais les mêmes requêtes, et le paramètre `random_seed` n'y change rien (1 question stable sur 3 pour le 8b, 0 sur 3 pour le 3b). D'où les requêtes enregistrées au moment de la mesure : rappeler le modèle après coup ne redonne pas ce que la mesure a vu.

Erreur commise : une provenance fausse

Les premières mesures de la réécriture ont été enregistrées sous le commit `b82a66a`, qui ne contenait pas encore la réécriture. Correctif : suffixe `+modifications` et provenance rétablie dans les fichiers existants. Plus tard, un message de commit a aussi cité une empreinte de golden set recopiée à la main et périmée ; le tableau du README est depuis généré directement depuis les fichiers de résultats.

Un document retiré du dossier reste indexé

Le livre avait été retiré du dossier corpus mais restait dans la base. Sur une première mesure, deux des quatre régressions de la réécriture venaient de lui. Il a finalement été conservé ; l'ingestion ne sait toujours pas supprimer un document orphelin.

14. La réécriture de requête

14.1 Le principe

Le corpus est en anglais, les questions en français. Quand le terme clé se ressemble dans les deux langues (Dijkstra, convexe/convex), la recherche s'en sort ; quand il ne se ressemble pas (« parcours en largeur » / *breadth-first search*), elle rate. Un petit modèle sert donc de traducteur terminologique : il ne voit aucun passage et produit seulement des requêtes de recherche.

```
« Comment fonctionne le parcours en largeur d'un graphe ? »
├── question d'origine
├── "breadth-first search algorithm graph traversal"
├── "queue data structure level-order traversal"
└── | un seul appel d'embeddings pour les 3
```

3 recherches hybrides (25 candidats chacune) → fusion RRF → 5 passages

14.2 Trois garde-fous

- **La question d'origine est toujours conservée** : une mauvaise reformulation ajoute un classement, elle ne remplace jamais celui qui marchait.
- **Une panne ne casse jamais /chat** : JSON invalide ou modèle injoignable, on retombe exactement sur la recherche simple.
- **Aucun exemple du golden set dans le prompt** : écrire « parcours en largeur → breadth-first search » en exemple soufflerait la réponse à la question d'évaluation qui échouait. Le score monterait sans rien mesurer. L'exemple vient d'un domaine hors des cours, et un test vérifie qu'aucun terme du golden set n'y figure.

Les appels se font à température 0 et en mode JSON (`response_format: json_object`), avec au plus deux reformulations. Le mécanisme a été sondé sur des questions **absentes** du golden set : ajuster le prompt en regardant les questions d'évaluation reviendrait à optimiser sur le jeu de test.

14.3 Les résultats

Huit mesures sur le golden set v2 (`7be5592d2ef6`), stratégies alternées dans le temps pour qu'une dérive éventuelle ne favorise aucune variante. 58 questions avec réponse attendue ; moyenne et [min–max].

Stratégie	Répét.	Hit@5	MRR	Précision@5	FR éloigné MRR	FR éloigné Préc.@5	Latence médiane
Hybride	2	0.966	0.908	0.803	0.783	0.640	333 ms
Réécriture 8b	3	1.000	0.988 [0.98–0.99]	0.912 [0.91–0.92]	0.967 [0.95–0.97]	0.877 [0.87–0.89]	1278 ms
Réécriture 3b	3	1.000	0.988 [0.98–0.99]	0.916 [0.91–0.92]	0.975	0.877 [0.87–0.89]	1044 ms

Ce qui est établi (étendues disjointes)

- La réécriture améliore **toutes les métriques de qualité**, avec les deux modèles ; le gain est le plus fort en français à termes éloignés (MRR 0.78 → 0.97, précision 0.64 → 0.88).
- Les deux échecs de l'hybride, à chaque répétition, disparaissent ; aucune question n'est instable d'une répétition à l'autre.
- Coût : environ 0,7 à 0,9 s de retrieval en médiane.
- **Entre 8b et 3b, seule la latence diffère** (3b plus rapide d'environ 230 ms) : aucune différence de qualité n'est établie.

14.4 La décision et son effet en production

Activation en production avec `ministral-3b-latest` : qualité équivalente au 8b, plus rapide, quota de 750 requêtes/minute au lieu de 188. Les réglages sont dans `fly.toml` (ce ne sont pas des secrets).

« Comment fonctionne le parcours en largeur d'un graphe ? »	Avant	Après
Réponse	Refus : « Je ne trouve pas la réponse... »	Explication du parcours en largeur (file FIFO, étapes)
Sources	Parcours en profondeur p.18, plus courts chemins ×3, coloration	Breadth first search p.1, p.11, p.5, p.13 , parcours en profondeur p.18
Durée totale	6,5 s	16,9 s

La durée totale augmente bien plus que les 0,7 s mesurés pour la réécriture. L'explication la plus probable est la génération — un refus de deux lignes contre une explication complète — éventuellement un réveil à froid juste après le déploiement. **Elle n'a pas été mesurée** : c'est précisément la question que l'étape suivante doit trancher.

15. Exploiter le projet au quotidien

15.1 Démarrer en local

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements-dev.txt
cp .env.example .env          # puis renseigner les vraies clés
docker compose up -d db
python run.py                  # sous Windows (boucle d'événements)
```

15.2 Aide-mémoire des commandes

Besoin	Commande
Documentation interactive	ouvrir http://localhost:8000/docs
Ingérer ou mettre à jour le corpus	<code>python -m app.ingest "C:/chemin/corpus"</code>
Réindexer tout	<code>python -m app.ingest "C:/chemin/corpus" --force</code>
Tests	<code>pytest -q</code>
Lint et format	<code>ruff check .</code> puis <code>ruff format --check .</code>
Mesurer le retrieval	<code>python -m eval.retrieval --nom X --strategie reecriture</code>
Synthétiser des répétitions	<code>python -m eval.synthese --groupe nom=r1,r2,r3</code>
Pile complète en conteneurs	<code>docker compose up --build</code>
Déployer	<code>fly deploy --remote-only</code>
Journaux de production	<code>fly logs</code>
État des machines	<code>fly status</code>
Une seule machine	<code>fly scale count 1 --app mini-copilot</code>
Secrets (noms seulement)	<code>fly secrets list</code>

15.3 Ajouter des documents au corpus

- Déposer les fichiers dans le dossier corpus, puis relancer l'ingestion : seuls les fichiers nouveaux ou modifiés sont traités.
- Pour la production, lancer l'ingestion avec `DATABASE_URL` pointant vers Neon (lu depuis `.env.prod`).
- Ajouter au golden set quelques questions sur le nouveau contenu, étiquetées en lisant le document, puis relancer les mesures.

15.4 Enrichir le golden set

- Écrire la question telle qu'un étudiant la poserait réellement.
- Trouver la réponse en lisant le document, jamais avec `/chat`.
- Choisir la catégorie selon la ressemblance du terme clé avec l'anglais.
- Lancer `pytest tests/test_golden_set.py` pour valider la structure.
- Mesurer à nouveau : l'empreinte du golden set change, les anciennes mesures ne sont plus comparables.

16. Dépannage

Symptôme	Cause probable	Action
502 avec LLM HTTP 429 ... quota	Quota à 0 pour ce modèle	Lire <code>x-ratelimit-limit-req-minute</code> ; changer de modèle
502 avec 403 not available	Modèle hors abonnement	Changer de modèle
502 avec ReadTimeout	Génération trop longue	Augmenter <code>REQUEST_TIMEOUT_SECONDS</code>
401 sur /chat	Header <code>X-API-Key</code> absent ou faux	Utiliser <code>API_KEY</code> du <code>.env</code>
Comportement qui ne correspond pas au code	Ancien serveur sur le même port	<code>netstat -ano findstr :8000</code>
L'app ne démarre pas : base injoignable	Base arrêtée ou mauvaise URL	<code>docker compose up -d db</code> ; vérifier <code>DATABASE_URL</code>
PoolTimeout	Type <code>vector</code> absent au démarrage du pool	Vérifier que <code>init_schema()</code> s'exécute avant <code>open_pool()</code>
ProactorEventLoop sous Windows	Boucle d'événements incompatible	Lancer <code>python run.py</code>
UnicodeEncodeError dans un script	Console Windows en cp1252	<code>PYTHONIOENCODING=utf-8</code>
Conteneur unhealthy en permanence	localhost résout en IPv6	Utiliser <code>127.0.0.1</code>
Beaucoup de Quota embeddings atteint	Plus de 60 embeddings/minute	Normal pendant une ingestion ou une mesure ; le back-off gère
Une question du cours non trouvée	Terme sans équivalent proche en anglais	Vérifier <code>REECRITURE_REQUETES=true</code> ; ajouter la question au golden set

17. Les leçons transversales

- **Mesurer au lieu de supposer.** Le seuil de distance « évident » ne fonctionnait pas ; la recherche vectorielle « suffisante » ratait les noms propres ; la réécriture a été activée sur des étendues, pas sur une impression.
- **Isoler avant de conclure.** Le diagnostic du quota a d'abord été faux faute d'avoir isolé la clé, le compte et le modèle.
- **Vérifier le vrai code de sortie.** Un pipeline shell ment : `| tail` a transformé un build raté en succès.
- **Une mesure sans provenance est invérifiable.** Commit, état de l'arbre, empreinte des étiquettes, requêtes générées : tout est enregistré.
- **Méfiance envers ses propres corrections.** Corriger une étiquette parce que le système la conteste biaise l'évaluation en sa faveur.
- **Pas d'optimisation sur le jeu de test.** Aucun exemple du golden set dans un prompt, et le prompt n'est jamais ajusté en regardant les questions d'évaluation.
- **Une panne se diagnostique depuis son message.** Type d'exception, code HTTP juste, marche à suivre.
- **Les secrets se protègent par la règle, pas par la vigilance.** Motifs `.gitignore` larges, vérification de l'index avant chaque commit.
- **Écrire ses erreurs.** Les erreurs de ce projet sont documentées dans le code, les commits et ce guide : c'est ce qui les rend utiles.

18. La suite

18.1 Étape 3 — Observabilité avec Langfuse (recommandée)

Elle vient maintenant pour une raison concrète : la réponse sur le parcours en largeur est passée de 6,5 s à 16,9 s et la cause n'a pu être que supposée. Langfuse enregistre chaque étape d'une requête sous forme de *trace*, avec durée, entrées et sorties.

Ce qu'il faudra construire

- **Une trace par requête** `/chat`, avec des étapes imbriquées : réécriture, embeddings, recherche SQL, génération.
- **Les métadonnées utiles** : requêtes générées, identifiants des passages retenus, modèle, tokens consommés, latence de chaque étape.
- **L'identifiant de trace renvoyé par** `/chat`, pour relier une réponse signalée à sa trace complète.
- **Les clés Langfuse dans les secrets** (Fly et `.env`), jamais dans le code.
- **Des tests qui restent sans réseau** : le traçage doit pouvoir être désactivé ou remplacé par un double.

Les choix à faire

Question	Options
Hébergement de Langfuse	Auto-hébergé via Docker Compose (les versions récentes demandent plusieurs services : base, stockage analytique, cache, stockage objet) ou offre cloud gratuite
Confidentialité	En cloud, les questions et les passages de cours partent chez un tiers : à décider consciemment
Échantillonnage	Tout tracer au début ; échantillonner si le volume grandit

Ce que l'étape permettra

- Mesurer enfin la répartition de la latence de `/chat`.
- Transformer les échecs réels des utilisateurs en questions du golden set.
- Suivre les tokens consommés, donc le coût, par requête.
- Rendre déboguable la future couche agent.

18.2 Les étapes suivantes

Étape	Contenu	Préalable
4b — Évaluation de la génération	Fidélité aux passages, pertinence, refus correct du hors-sujet	Un juge LLM plus capable que les modèles 3B/8B, ou une évaluation humaine sur un échantillon
5 — Agent	Enchaîner plusieurs étapes (décomposer une question, chercher plusieurs fois, vérifier) avec LangGraph	L'observabilité de l'étape 3
6 — Industrialisation	Déploiement continu, cache Redis des embeddings et réponses, modèle auto-hébergé (vLLM ou Ollama)	Dépôt distant pour activer la CI

18.3 Les limites connues à traiter

- **L'ingestion ne supprime pas les documents orphelins** : un fichier retiré du dossier reste indexé.
- **Le golden set est rédigé par le constructeur du système** : à enrichir avec de vraies questions de révision.
- **La latence de bout en bout n'est pas décomposée** : objet de l'étape 3.
- **Deux machines Fly** alors qu'une suffit pour un projet d'apprentissage.
- **Pas de dépôt distant** : la CI GitHub Actions est écrite mais n'a jamais tourné.
- **Avertissements CRLF de git** : un `.gitattributes` avec `* text=auto eol=lf` les supprimerait.
- **Les formules mathématiques sont mutilées** par l'extraction PDF : un extracteur plus fin ou de l'OCR mathématique améliorerait les cours de statistiques et d'optimisation.

Annexe A — Glossaire

Terme	Définition
RAG	Retrieval-Augmented Generation : retrouver des passages, puis les donner au modèle pour qu'il réponde à partir d'eux.
Embedding	Représentation d'un texte en vecteur de nombres, telle que deux textes de sens proche ont des vecteurs proches.
Chunk	Morceau de document indexé séparément (ici ~1200 caractères).
Distance cosinus	Mesure de l'écart d'orientation entre deux vecteurs ; opérateur <code><=></code> de pgvector.
HNSW	Index de recherche approchée des plus proches voisins, rapide sur de grands volumes.
tsvector / GIN	Représentation Postgres d'un texte pour la recherche par mots-clés, et l'index qui l'accélère.
Recherche hybride	Combinaison de la recherche vectorielle et de la recherche par mots-clés.
RRF	Reciprocal Rank Fusion : fusion de classements par la somme des $1/(K + \text{rang})$.
Réécriture de requête	Reformulation de la question par un modèle avant la recherche.
Golden set	Jeu de questions de référence dont la réponse attendue est connue, pour mesurer un système.
Hit@k	Part des questions dont au moins un passage pertinent est dans les k premiers.
MRR	Mean Reciprocal Rank : moyenne de $1/\text{rang}$ du premier passage pertinent.
Précision@k	Part des k passages renvoyés qui sont pertinents.
Back-off exponentiel	Réessayer après des délais qui doublent, pour laisser un quota se reconstituer.
Scale-to-zero	Arrêt des machines sans trafic, redémarrage à la demande.
Fail-closed	En cas de doute ou de configuration manquante, refuser plutôt qu'ouvrir.
Fuite (leakage)	Information du jeu d'évaluation qui influence le système évalué, et fausse la mesure.

Annexe B — Arborescence du dépôt

```

mini_copilot/
├── app/
│   ├── main.py      API FastAPI, authentification, codes HTTP
│   ├── llm.py       orchestration du RAG
│   ├── retrieval.py  recherche hybride, fusion RRF, multi-requêtes
│   ├── reecriture.py réécriture de requête
│   ├── embeddings.py vecteurs par lots, back-off
│   ├── mistral.py    client HTTP et appel chat
│   ├── parsers.py    lecture pdf / pptx / texte, normalisation
│   ├── chunking.py   découpage en passages
│   ├── ingest.py     pipeline d'ingestion
│   ├── db.py         schéma et pool Postgres
│   ├── config.py     configuration
│   └── boucle.py     correctif Windows
└── eval/

```

```
| | | golden_set.yaml 63 questions de référence
| | | metriques.py   Hit@k, MRR, Précision@k
| | | retrieval.py   exécution des mesures
| | | synthese.py    moyenne et étendue des répétitions
| | | resultats/     mesures enregistrées (JSON)
| | tests/          66 tests sans réseau
| | .github/workflows/ci.yml
| | Dockerfile
| | docker-compose.yml
| | fly.toml
| | run.py
| | requirements.txt / requirements-dev.txt
| | pyproject.toml
| | .env.example / .env.prod.example
| | README.md
```